TDA352/DIT352: Cryptography LP2, Fall 2026

Lecture Notes on

Pseudorandom Generators, Block Ciphers, and Modes of Operation

Lecturer: Elena Pagnin

Module 1, Lecture 2

Based on the Lecture 2 slides

Companion to the Lecture 1 notes

Contents

1 Pseudorandom Generators **1**
1.1 Stretching a Random Seed . . . 1
1.2 The Real-or-Random Security Game . . . 2
1.3 The One-Time PRG Cipher . . . 2

2 Single-Message Semantic Security **3**
2.1 The Left-or-Right Security Game . . . 3
2.2 Security of the One-Time PRG Cipher . . . 3

3 Block Ciphers **5**
3.1 Pseudorandom Permutations . . . 5
3.2 Block Cipher Syntax . . . 5
3.3 Indistinguishability Under Chosen Plaintext Attack . . . 6
3.4 The Advanced Encryption Standard (AES) . . . 7

4 Modes of Operation **7**
4.1 Electronic Code Book (ECB) . . . 7
4.2 Cipher Block Chaining (CBC) . . . 8
4.3 Counter Mode (CTR) . . . 8
4.4 Security of the Modes . . . 9
4.5 Message Padding and PKCS#7 . . . 9

1 Pseudorandom Generators

Lecture 1 established that the One-Time Pad (OTP) is perfectly secret when its uniform key is as long as the message and used only once. Shannon's theorem gives the constraint behind that key length: perfect secrecy and correctness require $|\mathcal{K}| \geq |\mathcal{M}|$. To use a shorter key, we replace perfect secrecy with *computational security*, which limits the attacker to efficient computation. As in Kerckhoffs' principle, the construction is public and only the secret key is hidden.

We write $x \xleftarrow{\$} S$ for a uniform draw from a finite set S, and $\oplus$ for bitwise XOR. An efficient adversary is a probabilistic polynomial-time (PPT) algorithm in the security parameter. A nonnegative function $\mathrm{negl}(n)$ is negligible if for every positive polynomial p there exists n_0 such that $\mathrm{negl}(n) < 1/p(n)$ for all $n \geq n_0$.

1.1 Stretching a Random Seed

Definition 1 (Pseudorandom Generator). A pseudorandom generator (PRG) is a deterministic, efficiently computable function

$$\mathsf{PRG} : \{0,1\}^n \longrightarrow \{0,1\}^\ell, \qquad \ell = \ell(n) > n,$$

whose output on a uniform seed $s \xleftarrow{\$} \{0,1\}^n$ is computationally indistinguishable from a uniform string of length ℓ. The next game specifies the meaning of indistinguishability.

A fixed seed always gives the same output. The randomness in the experiment comes from sampling the seed. The generator stretches a small amount of true randomness into a longer string that an efficient observer should be unable to distinguish from true randomness.

Lecture slides 3–5, 7. Negligibility is recalled from Lecture 1, slide 25.

1.2 The Real-or-Random Security Game

The challenger $\mathcal{C}$ selects a hidden bit b. The adversary $\mathcal{A}$ receives one string and decides whether it came from the generator or from the uniform distribution. The function PRG is public.

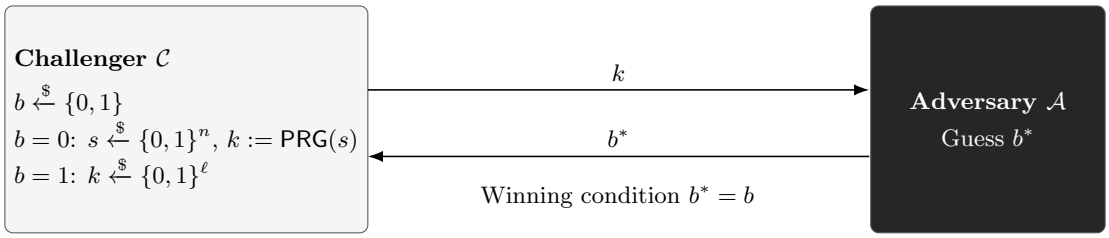


Figure 1: The PRG game. The challenge is a real generator output when $b = 0$ and a uniformly random string when $b = 1$.

Definition 2 (PRG Security). A PRG is secure if every PPT adversary $\mathcal{A}$ has negligible advantage in Figure 1:

$$\mathrm{Adv}_{\mathrm{PRG}}(\mathcal{A}) = \left|\Pr[b^* = b] - \tfrac{1}{2}\right| < \mathrm{negl}(n).$$

The probability is over the challenger's samples and the adversary's randomness.

The image is sparse. There are at most 2^n distinct PRG outputs, even when $\ell = 2n$ and the output space has 2^{2n} elements. In that case the image occupies at most a 2^{-n} fraction of the space. An adversary that enumerates all seeds can test membership in the image. It always accepts a real output and accepts a uniform string with probability at most 2^{-n}. This test can require 2^n evaluations, which is why sparsity is compatible with computational security. Slide 8 illustrates the work factor with $n \geq 128$ and a budget of 2^{80} evaluations.

1.3 The One-Time PRG Cipher

Definition 3 (One-Time PRG Cipher). Let PRG be a secure generator as above. Set $\mathcal{K} = \{0,1\}^n$ and $\mathcal{M} = \mathcal{C} = \{0,1\}^\ell$.

$$\begin{aligned} \mathsf{KeyGen}(1^n) &\quad \text{Sample } s \xleftarrow{\$} \{0,1\}^n \text{ and output } s. \\ E(s,m) &\quad \text{Output } c := \mathsf{PRG}(s) \oplus m. \\ D(s,c) &\quad \text{Output } m := \mathsf{PRG}(s) \oplus c. \end{aligned}$$

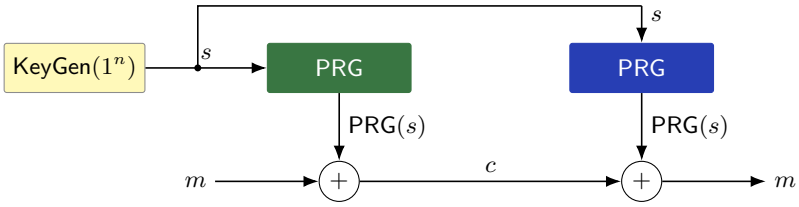


Figure 2: The one-time PRG cipher. Sender and receiver share the seed s, expand it to the same pad, and XOR with the plaintext or ciphertext respectively.

Correctness. XORing the same pad twice cancels it: $D(s, E(s,m)) = \mathsf{PRG}(s) \oplus \mathsf{PRG}(s) \oplus m = m$. The scheme uses a shorter key than the OTP. Since $|\mathcal{K}| = 2^n < 2^\ell = |\mathcal{M}|$, Shannon's theorem rules out perfect secrecy.

One-time use. If the same seed encrypts m_0 and m_1, the pad repeats, so $c_0 \oplus c_1 = m_0 \oplus m_1$, just as when an OTP key is reused. We first analyze a single challenge ciphertext under a freshly sampled secret seed.

Lecture slides 7–9.

2 Single-Message Semantic Security

2.1 The Left-or-Right Security Game

We now ask whether an attacker can tell which of two messages it chose was encrypted. In the *left-or-right* experiment, the challenger encrypts the left message when $b = 0$ and the right message when $b = 1$. The adversary knows the symmetric encryption algorithms (KeyGen, E, D) and chooses both candidates before seeing the challenge.

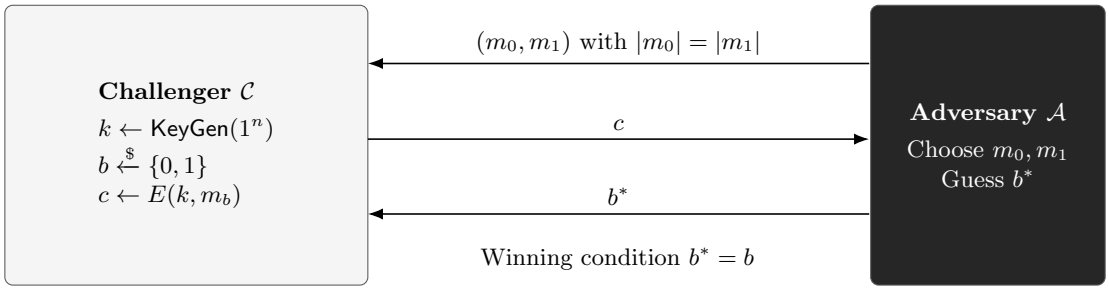


Figure 3: Single-message semantic security. The attacker sees one encryption of a message from its chosen pair.

Definition 4 (Single-Message Semantic Security). A symmetric encryption scheme is semantically secure in the single-message sense if for every PPT adversary $\mathcal{A}$ in Figure 3,

$$\mathrm{Adv}_{\mathrm{sem.sec.}}(\mathcal{A}) = \left|\Pr[b^* = b] - \tfrac{1}{2}\right| < \mathrm{negl}(n).$$

The probability is over key generation, the hidden bit, any encryption randomness, and the adversary's randomness.

Equal lengths. The security game allows the ciphertext to reveal the plaintext length. Requiring $|m_0| = |m_1|$ prevents the adversary from deciding the game by comparing lengths alone.

Baseline guesses. A uniform random guess succeeds with probability $1/2$. A guess that is always 0 or always 1 also succeeds with probability $1/2$, since b is uniform. Each strategy has advantage zero.

The OTP in this game. With a fresh uniform pad k, for each fixed message m_b the map $k \mapsto k \oplus m_b$ is a bijection. The challenge ciphertext is uniform for either value of b. Every adversary, including an unbounded one, therefore has advantage zero.

2.2 Security of the One-Time PRG Cipher

Theorem 1 (Secure Encryption with Short Keys). If $\mathsf{PRG} : \{0,1\}^n \to \{0,1\}^\ell$ is a secure pseudorandom generator, then the one-time PRG cipher of Definition 3 is semantically secure in the single-message game.

The proof uses a *reduction*. Given any efficient attacker $\mathcal{A}$ for the encryption game, we build an efficient attacker $\mathcal{A}'$ for the PRG game. The same challenge simulation will yield real PRG encryption in one branch and a perfectly secret OTP in the other. The lecture presents the reduction as a worked example and marks it as outside the exam scope.

Lecture slides 10–14.

Proof of Theorem 1. Use b for the PRG challenger's hidden bit, b' for the hidden bit of the encryption game simulated by $\mathcal{A}'$, b^* for $\mathcal{A}$'s guess, and $\widetilde{b}$ for the guess $\mathcal{A}'$ returns to its challenger.

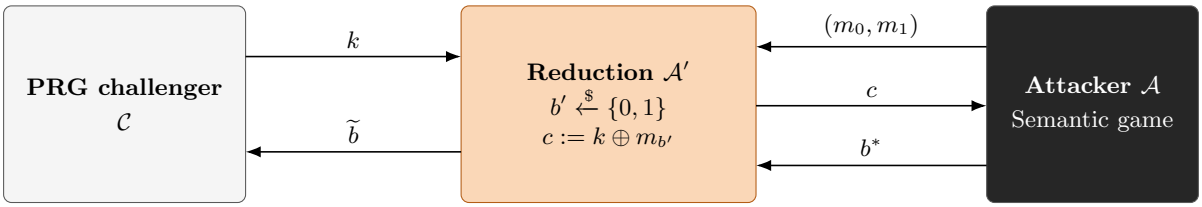


Figure 4: The reduction. The attacker $\mathcal{A}'$ uses the PRG challenge string k as the pad in the simulated left-or-right game.

Proof. The PRG challenger gives $\mathcal{A}'$ a string k that is either $\mathsf{PRG}(s)$ for a uniform seed s or uniform over $\{0,1\}^\ell$. The reduction runs $\mathcal{A}$ to get (m_0, m_1), samples $b' \xleftarrow{\$} \{0,1\}$ independently, and supplies $c := k \oplus m_{b'}$ as the challenge ciphertext. Once $\mathcal{A}$ returns b^*, the reduction outputs

$$\widetilde{b} = \begin{cases} 0 & \text{if } b^* = b', \\ 1 & \text{if } b^* \neq b'. \end{cases}$$

Case $b = 0$ (real pad). The challenge is distributed exactly as a real one-time PRG encryption of $m_{b'}$. Let p be the probability that $\mathcal{A}$ wins the semantic security game. Since the reduction wins exactly when $b^* = b'$, its conditional success probability is $\Pr[\mathcal{A}' \text{ wins} \mid b = 0] = p$.

Case $b = 1$ (uniform pad). The simulation is an OTP with a uniform independent pad. Thus b^* equals b' with probability $1/2$. In this branch the reduction wins when $b^* \neq b'$, which also has probability $1/2$.

By the law of total probability and $\Pr[b = 0] = \Pr[b = 1] = 1/2$,

$$\begin{aligned} q := \Pr[\mathcal{A}' \text{ wins}] &= \Pr[\mathcal{A}' \text{ wins} \mid b = 0] \Pr[b = 0] + \Pr[\mathcal{A}' \text{ wins} \mid b = 1] \Pr[b = 1] \\ &= \tfrac{1}{2}p + \tfrac{1}{4}. \end{aligned}$$

Rearranging gives $p = 2q - \frac{1}{2}$. Hence

$$\mathrm{Adv}_{\text{sem.sec.}}(\mathcal{A}) = \left|p - \tfrac{1}{2}\right| = 2\left|q - \tfrac{1}{2}\right| = 2\,\mathrm{Adv}_{\mathrm{PRG}}(\mathcal{A}').$$

The simulation uses one run of $\mathcal{A}$, one random bit, and an XOR of ℓ bits. Thus $\mathcal{A}'$ is PPT when $\mathcal{A}$ is PPT. A non-negligible advantage for $\mathcal{A}$ would yield half that advantage against the PRG, contradicting its assumed security. □

What remains. The reduction protects one ciphertext under one seed. To encrypt many messages we need a scheme that can keep a shared key while changing the pad. We also need to accommodate different message lengths. Integrity and how parties establish a shared secret are separate topics, taken up in Lecture 4 and Module 2 respectively.

Lecture slides 15–19.

3 Block Ciphers

One approach to getting a new pad for every message is to draw a fresh seed s and send $(s, \mathsf{PRG}(s) \oplus m)$. This reveals the pad: anyone can evaluate the public generator at the transmitted seed and XOR the result with the second component. The lecture instead makes the pad depend on a shared key, while keeping the input s public.

3.1 Pseudorandom Permutations

Let $\mathcal{M} = \{0,1\}^n$. A *random permutation* is a bijection $\pi : \mathcal{M} \to \mathcal{M}$ sampled uniformly from the set $\mathrm{Perms}(\mathcal{M})$ of all such bijections. The ideal object preserves distinct inputs as distinct outputs.

Definition 5 (Pseudorandom Permutation, Informal). A pseudorandom permutation (PRP) is a keyed family $\{f_k : \{0,1\}^n \to \{0,1\}^n\}_{k \in \{0,1\}^\kappa}$ of efficiently computable permutations. An efficient algorithm with oracle access to the mapping should have only negligible advantage in distinguishing f_k for a uniform hidden k from an independently sampled uniform permutation π on $\{0,1\}^n$.

The ideal comparison here is a random permutation, as in the lecture's mode-security statement on slide 44. An arbitrary random function can map distinct inputs to the same output.

Counting permutations. For 2^n inputs there are 2^n choices for the first image, $2^n - 1$ for the second, and so on. Thus

$$\left|\mathrm{Perms}(\mathcal{M})\right| = (2^n)!, \qquad \left|\{f_k : k \in \{0,1\}^K\}\right| \le 2^K.$$

For $n = 3$ the first count is $8! = 40\,320$, roughly 2^{15}, and is small enough to enumerate. The lecture illustrates three possible maps with the identity, bitwise inversion, and a scramble. For realistic block sizes a uniform permutation is far too large to store explicitly. A keyed cipher uses a compact key to select an efficiently evaluable permutation that should look random to an efficient observer.

A candidate for multi-message encryption. Generate $k \xleftarrow{\$} \{0,1\}^K$ once. For each n-bit message choose a fresh random $s \xleftarrow{\$} \{0,1\}^n$ and return $c = (s, f_k(s) \oplus m)$. On input $c = (s,t)$ the receiver outputs $t \oplus f_k(s) = m$. The transmitted s allows the receiver to reproduce the pad with the shared key. A repeated s under the same key repeats the pad. The lecture states security from the PRP assumption for this construction with fresh inputs. Section 3.3 defines the multi-message security goal.

3.2 Block Cipher Syntax

Definition 6 (Block Cipher). A block cipher has key space $\mathcal{K} = \{0,1\}^K$ and block space $\mathcal{M} = \{0,1\}^n$. Encryption $E : \mathcal{K} \times \mathcal{M} \to \mathcal{M}$ is efficient and deterministic. For every $k \in \mathcal{K}$, $E(k,\cdot)$ is a permutation and its inverse $D(k,\cdot)$ is efficient and deterministic:

$$D(k, E(k,m)) = m \qquad \text{for every } k \in \mathcal{K},\ m \in \mathcal{M}.$$

Both plaintext and ciphertext inputs to the respective maps are called blocks. The key length K and the block length n are separate parameters.

Security versus syntax. Invertibility gives correctness. The security assumption is that the keyed permutation behaves like a random permutation to an efficient observer. The mode of operation determines how to use this primitive on longer messages and how to encrypt repeatedly under the same key.

Lecture slides 21–26.

3.3 Indistinguishability Under Chosen Plaintext Attack

The attacker can sometimes cause a system to encrypt a message of its choosing. The lecture gives two examples. The British military could place mines at selected locations in the hope that the Germans would send encrypted reports about those locations. In a web setting, attacker-controlled JavaScript can cause a victim client to make an HTTPS connection.

Adversarial goal. Distinguish the encryption of one known message from the encryption of another. Even a small advantage over random guessing can contradict the security definition.

Adversarial power. The attacker is PPT, observes the communication channel, knows the encryption scheme, and can request encryptions of messages of its choice. The key is shared by all oracle calls and the challenge, and remains secret. The queries are *adaptive*: each m_i can depend on earlier responses.

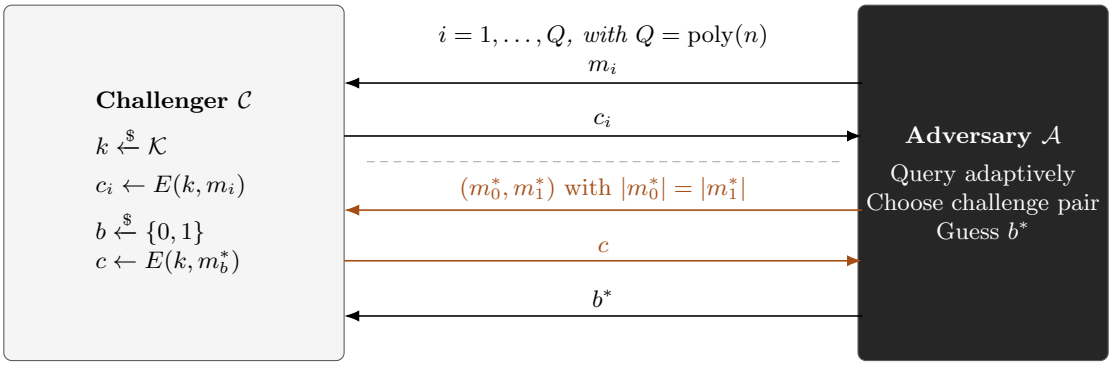


Figure 5: The IND-CPA game as presented in the lecture. Adaptive encryption queries precede the single left-or-right challenge, shown in orange.

Definition 7 (IND-CPA Security). An encryption scheme is indistinguishable under chosen plaintext attack if every PPT adversary $\mathcal{A}$ in Figure 5 has negligible advantage

$$\text{Adv}_{\text{IND-CPA}}(\mathcal{A}) = \left|\Pr[b^* = b] - \tfrac{1}{2}\right| < \text{negl}(n).$$

Each encryption call uses its own randomness if the scheme is randomized.

An adversary may ignore the encryption oracle, recovering the single-message game as a special case. Conversely, the oracle lets an attacker compare a challenge with a previous encryption under the same key. In a deterministic scheme with two distinct equal-length messages, it can request $E(k, m_0)$, challenge on (m_0, m_1), then test equality with its earlier result. Correctness makes the ciphertexts for distinct messages different, so the attack wins every time.

The role of randomization. A randomized encryption algorithm can encrypt the same message twice under the same key and produce different ciphertexts. The mode must arrange this variability while preserving correct decryption. The next section shows how a public IV or nonce can do so.

Lecture slides 27–29, 43.

3.4 The Advanced Encryption Standard (AES)

AES is the lecture's main example of a block cipher. The ten-round sketch is AES-128. It encrypts a 128-bit block M under a 128-bit key K. The key expansion derives round keys $K_0, \ldots, K_{10}$ from K. Each key addition XORs a round key into the state, and the other transformations provide substitution and mixing between key additions.

AES-128 encryption, following the lecture's round sketch

$(K_0, \ldots, K_{10}) \leftarrow \mathsf{Expand}(K)$
$s \leftarrow M \oplus K_0$
for $r = 1, \ldots, 10$ **do**
    $s \leftarrow \mathsf{SubBytes}(s)$
    $s \leftarrow \mathsf{ShiftRows}(s)$
    if $r \leq 9$ **then** $s \leftarrow \mathsf{MixColumns}(s)$
    $s \leftarrow s \oplus K_r$
return s

Round transformations. SubBytes acts on bytes. Slide 30 calls it `substitute-nibbles`. ShiftRows cyclically shifts the state rows. MixColumns mixes bytes within each column and is omitted in the last round. The XOR step is AddRoundKey. The round keys are derived from the same key, so the key additions use related round keys. See NIST FIPS 197, §5, for the AES terminology and specification [3].

Avalanche effect. A one-bit change in a plaintext block or in the key should change roughly half the output bits, in an unpredictable pattern. Changing all the bits would give a predictable complement. The avalanche effect is a design goal. The lecture's security assumption is indistinguishability from a random permutation.

Key size. A K-bit key space contains 2^K candidates. An attacker can test candidates against known plaintext–ciphertext pairs. The lecture's rule of thumb is that a budget of 2^t trials requires more than t key bits to put full enumeration outside that budget. Key length sets the exhaustive-search work factor, while the construction determines whether faster attacks exist.

Other block ciphers mentioned. PRESENT targets constrained devices, Camellia uses 128-bit blocks and 128-, 192-, or 256-bit keys, SM4 uses 128-bit blocks and keys, and Triple-DES appears as a legacy example.

Lecture slides 30–33.

4 Modes of Operation

A block cipher operates on one n-bit block at a time. A mode specifies how to encrypt $m = (m_0, \ldots, m_{v-1})$ with one key k, where each m_i is an n-bit block. We use c_i for the corresponding ciphertext blocks. A mode may also transmit a public value alongside them.

4.1 Electronic Code Book (ECB)

ECB applies the block cipher separately to every block. For $i = 0, \ldots, v-1$,

$$c_i = E(k, m_i), \qquad m_i = D(k, c_i).$$

Both directions are easy to implement and parallelizable. The same plaintext block always maps to the same ciphertext block under a fixed key, exposing repeated patterns within and across messages. Section 4.4 gives the lecture's explicit attack on AES-ECB.

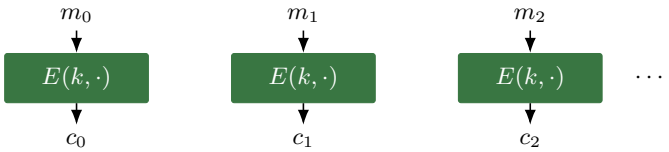


Figure 6: ECB encryption. Each block is encrypted independently with the same key.

Lecture slides 35–36.

4.2 Cipher Block Chaining (CBC)

CBC XORs each plaintext block with the preceding ciphertext block before applying E. A fresh, independently uniform initialization vector $IV \xleftarrow{\$} \{0,1\}^n$ supplies the first input to the chain and travels with the ciphertext. Set $c_{-1} := IV$ and output $(IV, c_0, \ldots, c_{v-1})$. For $0 \le i < v$,

$$c_i = E(k, m_i \oplus c_{i-1}), \qquad m_i = D(k, c_i) \oplus c_{i-1}.$$

In particular, $c_0 = E(k, m_0 \oplus IV)$ and $m_0 = D(k, c_0) \oplus IV$. Applying $D(k, \cdot)$ removes the block cipher and a second XOR removes c_{i-1}.

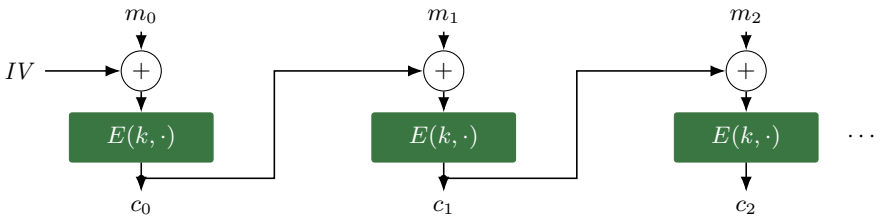


Figure 7: CBC encryption. Each ciphertext block feeds the XOR for the following block. A circled plus represents bitwise XOR.

Parallelism. Encryption is sequential because c_i must be available to form c_{i+1}. During decryption every c_i and its predecessor are already available, so decryption can run in parallel.

IV reuse. Two messages encrypted with the same key and IV have identical ciphertexts through their common prefix of whole blocks. Once their first different plaintext blocks enter the same chaining value, those ciphertext blocks differ by injectivity. Figure 9 illustrates the resulting prefix leak.

4.3 Counter Mode (CTR)

CTR turns the block cipher into a pad generator. Let N be the public nonce and ctr_i the encoded counter for block i. Their concatenation $N \,\|\, \mathrm{ctr}_i$ is exactly n bits. The slide illustrates counters starting at 0 and increasing by one. Transmit N with the ciphertext and use

$$p_i := E(k, N \,\|\, \mathrm{ctr}_i), \qquad c_i = m_i \oplus p_i, \qquad m_i = c_i \oplus p_i.$$

Both operations use forward encryption E and XOR. Each pad block can be computed independently, so encryption and decryption are parallelizable. The receiver needs the nonce and the counter value for each block.

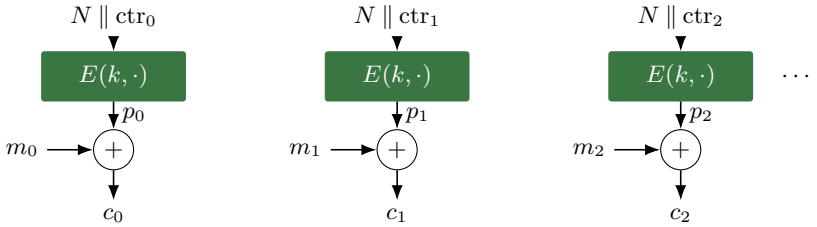


Figure 8: CTR encryption. Distinct nonce–counter inputs supply separate pad blocks.

Nonce and counter reuse. Never repeat a nonce–counter input under one key. A nonce is a number used once for that key, and the counter must stay within its allotted range. If an input repeats, both ciphertext blocks use the same pad p_i. Their XOR is $(m_i \oplus p_i) \oplus (m'_i \oplus p_i) = m_i \oplus m'_i$, the same cancellation that breaks OTP reuse. CTR can handle a final partial block by taking only the needed pad bits.

Lecture slides 37–39, 43.

4.4 Security of the Modes

Theorem 2 (ECB Fails IND-CPA). AES-ECB is insecure even in the single-message left-or-right game for two-block messages.

Proof. Choose distinct blocks B_0, B_1 and send $(m_0^*, m_1^*) = ((B_0, B_1), (B_1, B_1))$. On challenge ciphertext (c_0, c_1), guess $b^* = 1$ if $c_0 = c_1$ and guess $b^* = 0$ otherwise. If $b = 0$, the inputs are distinct and the permutation $E(k, \cdot)$ gives distinct outputs. If $b = 1$, the inputs are equal and the outputs coincide. Thus $\Pr[b^* = b] = 1$ and the advantage is $1/2$ in the lecture's convention. No preliminary encryption queries are needed. □

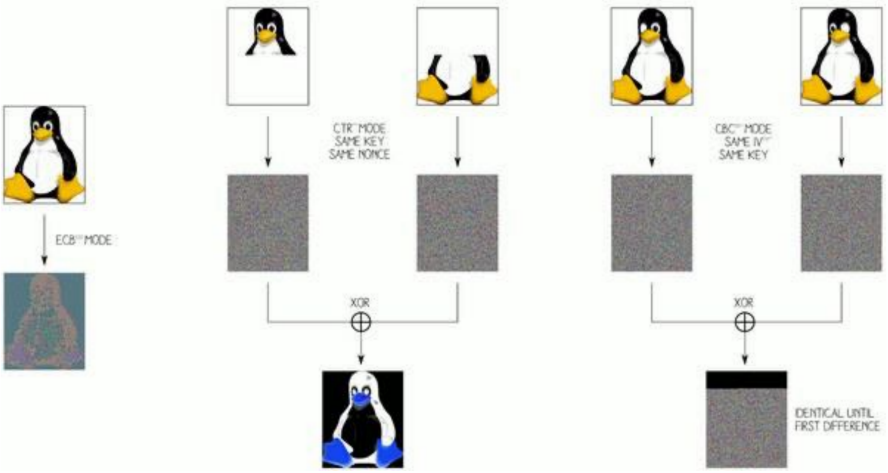


Figure 9: The lecture's visual examples (slide 39). Left, ECB preserves a visible pattern. Centre, XORing two CTR ciphertexts made with the same key and nonce exposes the XOR of the plaintexts. Right, reusing a CBC IV and key exposes the common prefix, visible as zeros in the ciphertext XOR until the first difference.

Theorem 3 (Security of CBC and CTR, as Stated in the Lecture). With a secure block cipher, CBC with fresh random IVs and CTR with distinct nonce–counter inputs give IND-CPA security under their respective conditions. The lecture states these results and skips their formal proofs.

Here a secure block cipher means one that an efficient adversary cannot distinguish from a uniform random permutation, using the appropriate oracle access. The result is conditional on that assumption for AES. A transmitted IV or nonce expands the ciphertext, as can padding. In settings where a unique input is supplied by the surrounding protocol, it need not be sent with every ciphertext.

4.5 Message Padding and PKCS#7

ECB and CBC take whole blocks. For a block length of B bytes, PKCS#7 appends N bytes, each encoding the integer N, where

$$N = B - (|m|_{\text{bytes}} \bmod B), \qquad 1 \le N \le B.$$

For instance, four missing bytes are filled with `04 04 04 04`. A message already aligned receives an entire block of padding. This allows unique removal. Simply appending zeros is ambiguous if the original message itself ends in zeros.

To remove padding, read the last byte as N. Check that N is in range and that all N final bytes have that value. Remove precisely those bytes on success and abort the protocol if the padding is invalid.

Lecture slides 39, 41–45.

References

[1] Elena Pagnin. *TDA352/DIT352 Cryptography, Module 1, Lecture 2.* Course slides.

[2] Elena Pagnin. *Module 1, Lecture 1.* Course slides. See also the companion *Course Introduction, Perfect Security, OTP* lecture notes.

[3] NIST. *Advanced Encryption Standard (AES)*, FIPS 197, updated 2023. doi:10.6028/NIST.FIPS.197-upd1.